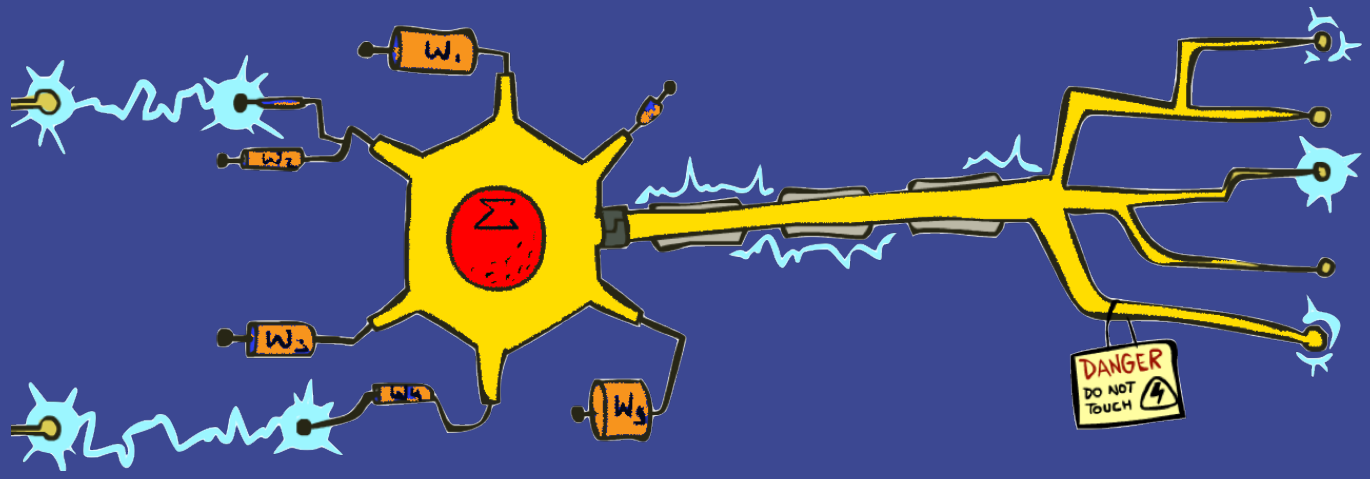
CS 156
Intro to AI



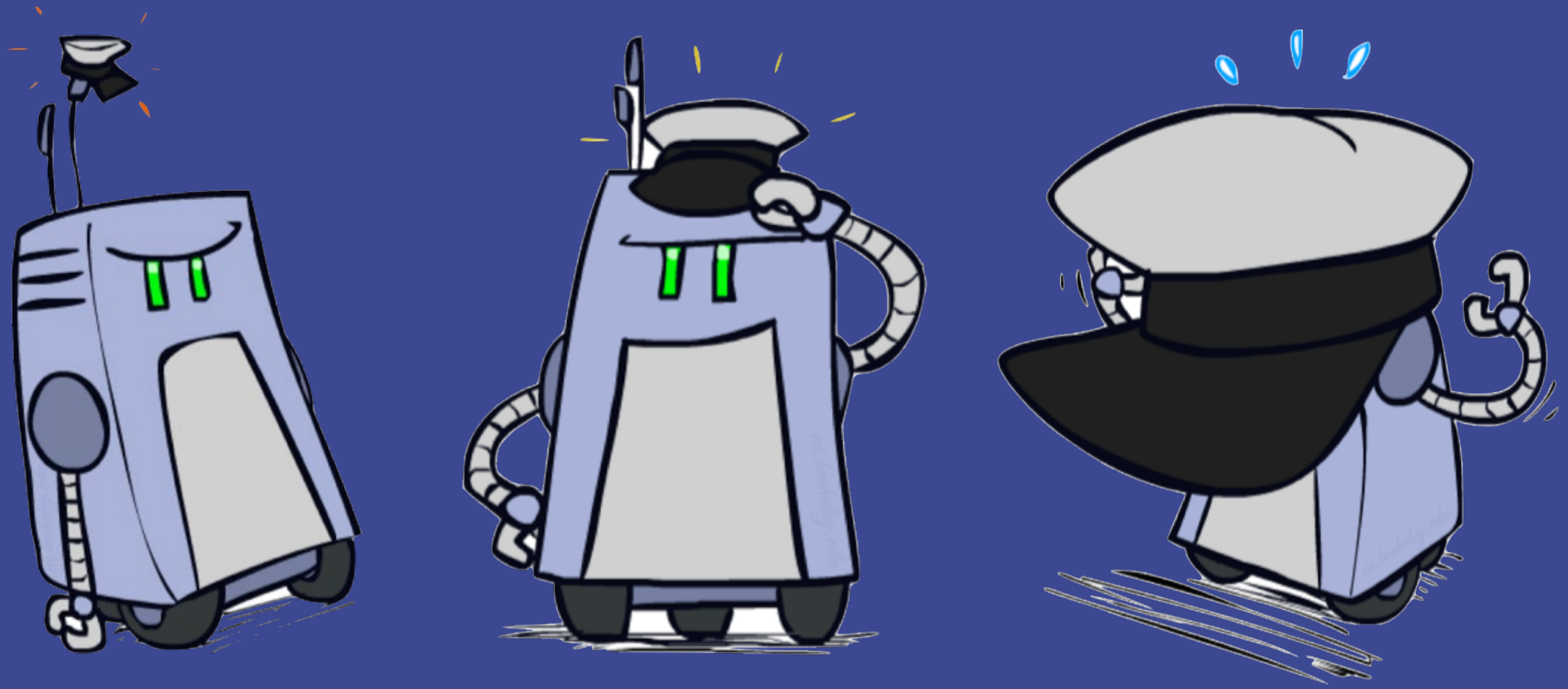
Naïve Bayes: Tuning Perceptron

These slides are based on the slides created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.
The artwork is by Ketrina Yim

Today

- Overfitting and Generalization
- Smoothing
- Tuning
- Perceptron

Generalization and Overfitting



Example: Overfitting

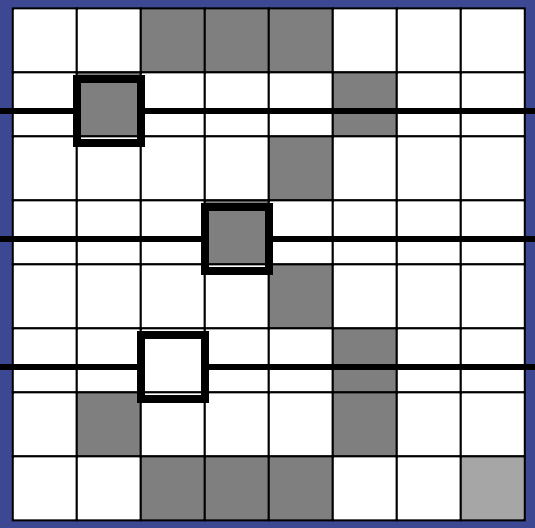
P(C=2) = 0.1

P(C=3) = 0.1

P(on|C=2) = 0.8

P(on|C=2) = 0.1

P(off|C=2) = 0.1



P(on|C=3) = 0.8

P(on|C=3) = 0.9

P(off|C=3) = 0.7

Based on the evidence so far, this image will most probably be classified as a:

A. 2

B. 3

Example: Overfitting

P(C=2) = 0.1

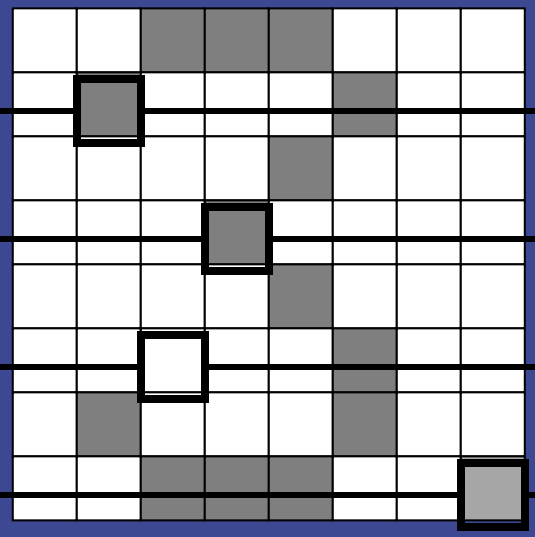
P(C=3) = 0.1

P(on|C=2) = 0.8

P(on|C=2) = 0.1

P(off|C=2) = 0.1

P(on|C=2) = 0.01



P(on|C=3) = 0.8

P(on|C=3) = 0.9

P(off|C=3) = 0.7

P(on|C=3) = 0.0

Based on the evidence so far, this image will most probably be classified as a:

A. 2

B. 3

Example: Overfitting

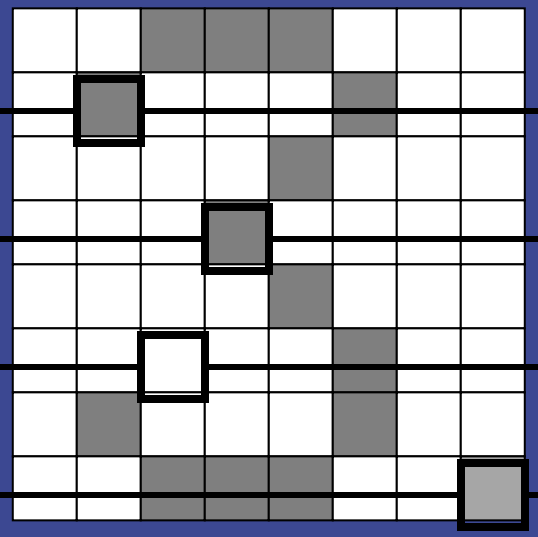
P(C=2) = 0.1

P(on|C=2) = 0.8

P(on|C=2) = 0.1

P(off|C=2) = 0.1

P(on|C=2) = 0.01



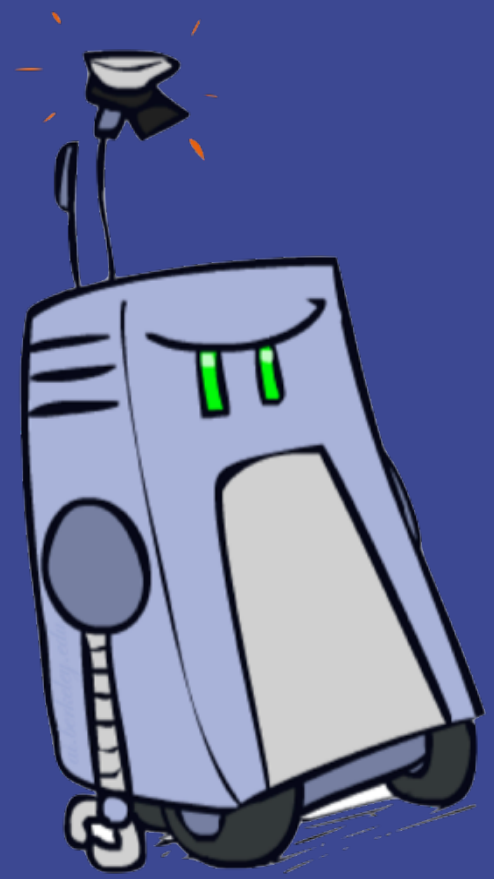
P(C=3) = 0.1

P(on|C=3) = 0.8

P(on|C=3) = 0.9

P(off|C=3) = 0.7

P(on|C=3) = 0.0



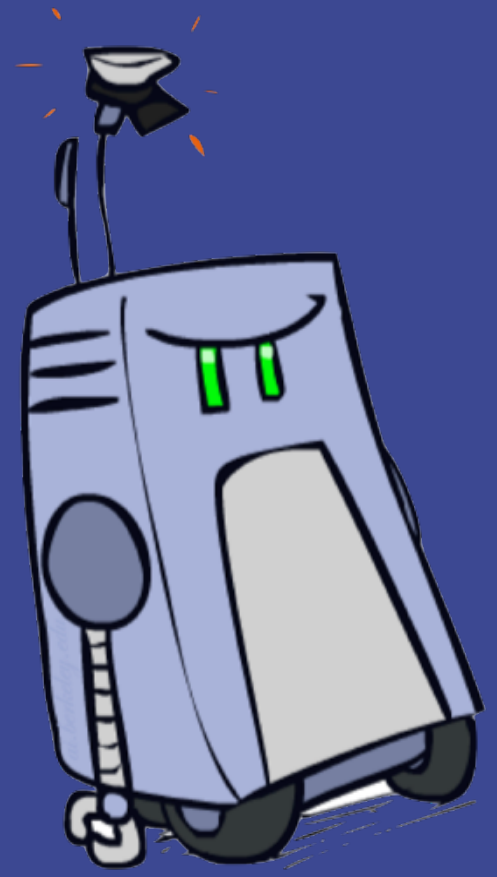
2 wins!!

Example: Overfitting

- Posteriors determined by *relative* probabilities (odds ratios):

$$\frac{P(W|ham)}{P(W|spam)} \qquad \frac{P(W|spam)}{P(W|ham)}$$

south-west : inf	screens : inf
nation : inf	minute : inf
morally : inf	guaranteed : inf
nicely : inf	$205.00 : inf
extent : inf	delivery : inf
seriously : inf	signature : inf
…	…



What went wrong here?

Generalization and Overfitting

Relative frequency parameters will overfit the training data!

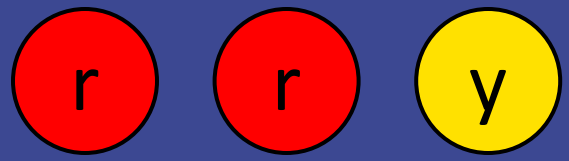
- Just because we never saw a 3 with pixel (15,15) on during training doesn’t mean we won’t see it at test time
- Unlikely that every occurrence of "minute" is 100% spam
- Unlikely that every occurrence of "seriously" is 100% ham
- What about all the words that don’t occur in the training set at all?
- In general, we can’t go around giving unseen events zero probability

Generalization and Overfitting

- As an extreme case, imagine using the entire email as the only feature
  - Would get the training data perfect (deterministic labeling)
  - Wouldn't *generalize* at all
  - Just making the bag-of-words assumption gives us some generalization, but isn't enough
- To generalize better: we need to smooth or regularize the estimates

Laplace Smoothing

- Laplace's estimate:
  - Pretend we saw every outcome once more than we actually did

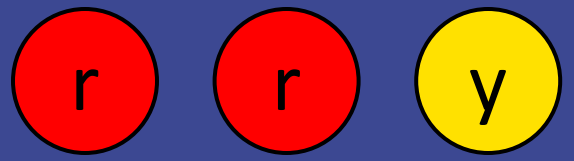


$P_{ML}(X) = [2/3, 1/3]$

$P_{LAP}(X) = [3/5, 2/5]$

Laplace Smoothing

- Laplace's estimate:
  - Pretend we saw every outcome once more than we actually did



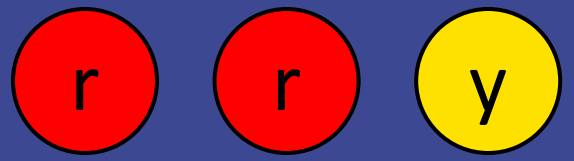
$P_{ML}(X) = [2/3, 1/3]$

$P_{LAP}(X) = [3/5, 2/5]$

- $P_{LAP}(x) = \frac{c(x)+1}{\sum_x [c(x)+1]} = \frac{c(x)+1}{N+|X|}$

Laplace Smoothing

- Laplace's estimate:
  - Pretend we saw every outcome k extra times
  - $P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$
  - What's Laplace with k = 0?



$P_{LAP,0}(X) = [2/3, 1/3]$

$P_{LAP,\,1}(X) = [3/5, 2/5]$

$P_{LAP,\,100}(X) = [102/203, 101/203]$

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

iClicker:

What is $P_{LAP,\,0}(p)$?

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

What is $P_{LAP,\,0}(p)$?

$P_{LAP,\,0}(p) = 1/5 = 0.2$

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

iClicker:

What is $P_{LAP,\,1}(p)$?

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

What is $P_{LAP,\,1}(p)$?

$P_{LAP,\,1}(p) = 1+1\,/(5+4) = 0.22$

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

iClicker:

What is $P_{LAP,\,2}(p)$?

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

$$P_{LAP,\,2}(p) = \frac{1+2}{5+2x4} = \frac{3}{5+8} = \frac{3}{13} = 0.23$$

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

iClicker:

What is $P_{LAP,100}(p)$?

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

$$P_{LAP,\,100}(p) = \frac{1+100}{5+100x4} = \frac{101}{405} = 0.249$$

Laplace Smoothing

$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

We have 4 colors: r, o, y, p.

iClicker:

What is $P_{LAP,\,1}(p)$?

Laplace Smoothing

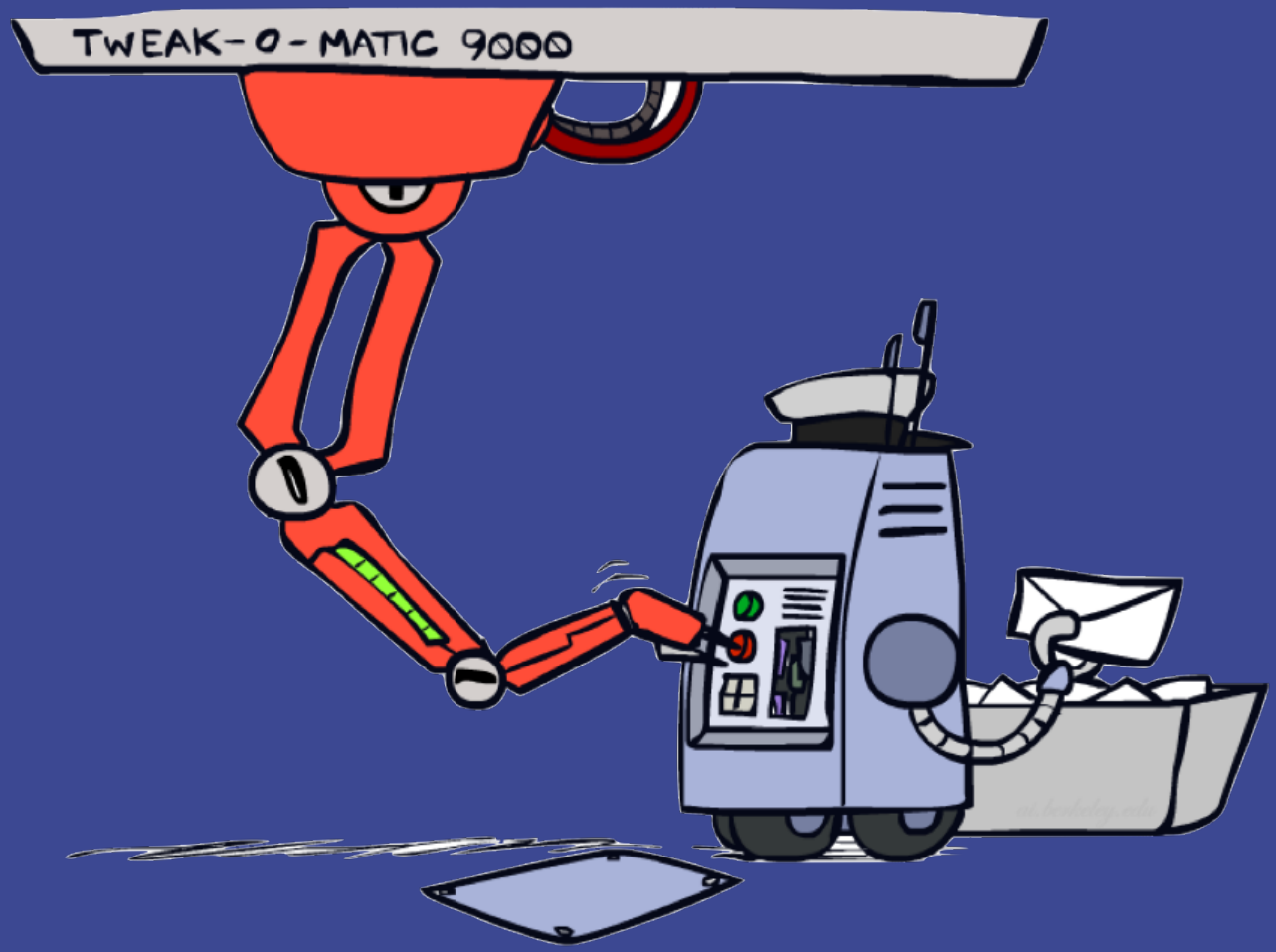
$$P_{LAP,\,k}(x) = \frac{c(x)+k}{N+k\,|X|}$$

We have 4 colors: r, o, y, p.

What is $P_{LAP,\,1}(p)$?

$P_{LAP,\,1}(p) = (0 + 1) / (5 + 4) = 0.11$

Tuning



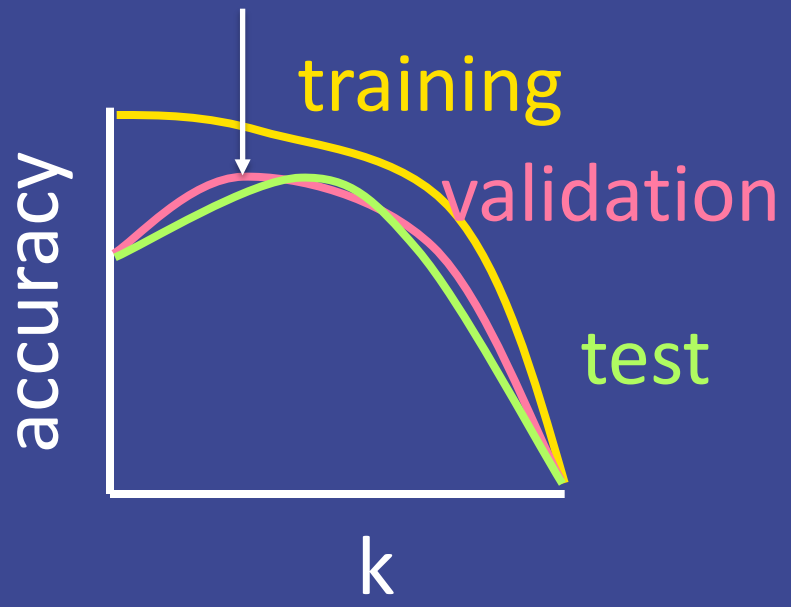
Tuning on Validation Data

- Now we have two kinds of unknowns
  - Parameters: the probabilities P(F|Y), P(Y)
  - Hyperparameters: the amount / type of smoothing to do (k)

Tuning on Validation Data

What should we learn where?

- Learn parameters from training data
- Tune hyperparameters on different data
- For each value of the hyperparameters (k), test on the held-out / validation data
- Choose the best value (best accuracy)
- Do a final test on the test data



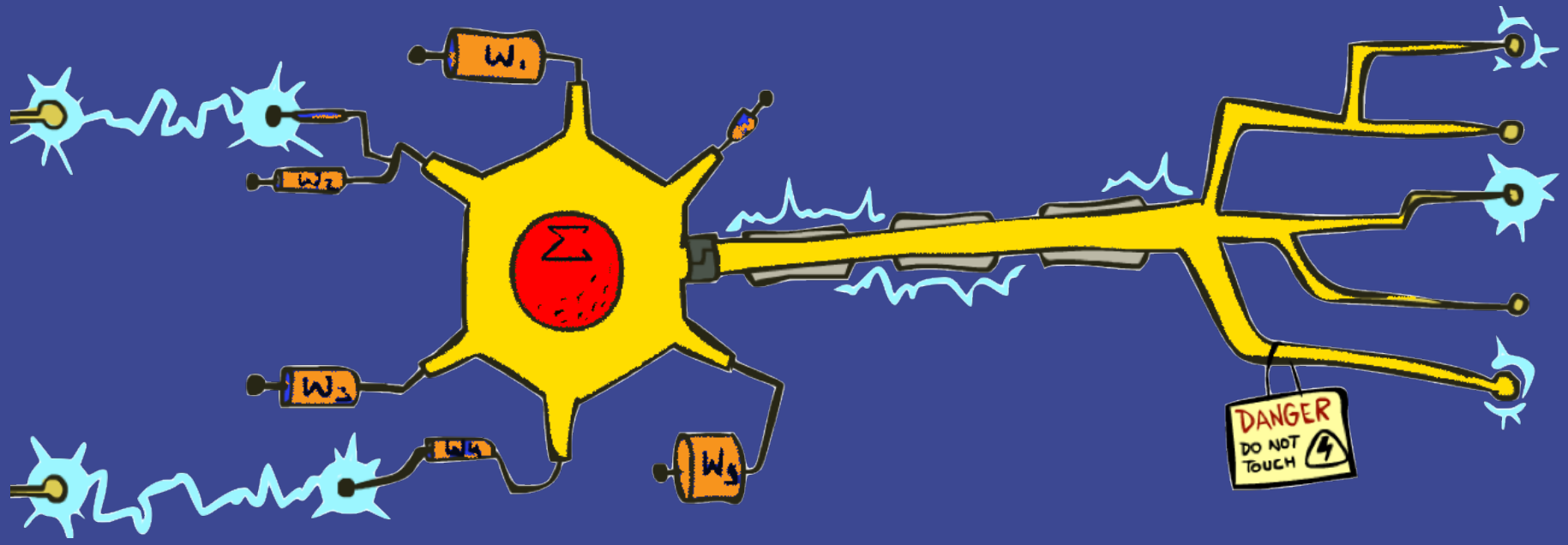
Baselines

- Important first step: get a baseline
  - Help determine how hard the task is
  - Help know what a "good" accuracy is
- Weak baseline: most frequent label classifier
  - Gives all test instances the most common label in the training set
  - For spam, label everything as ham => 66% accuracy
  - A spam classifier that gets 70% is not very good…
- For real research, usually use previous work as a (strong) baseline

Summary

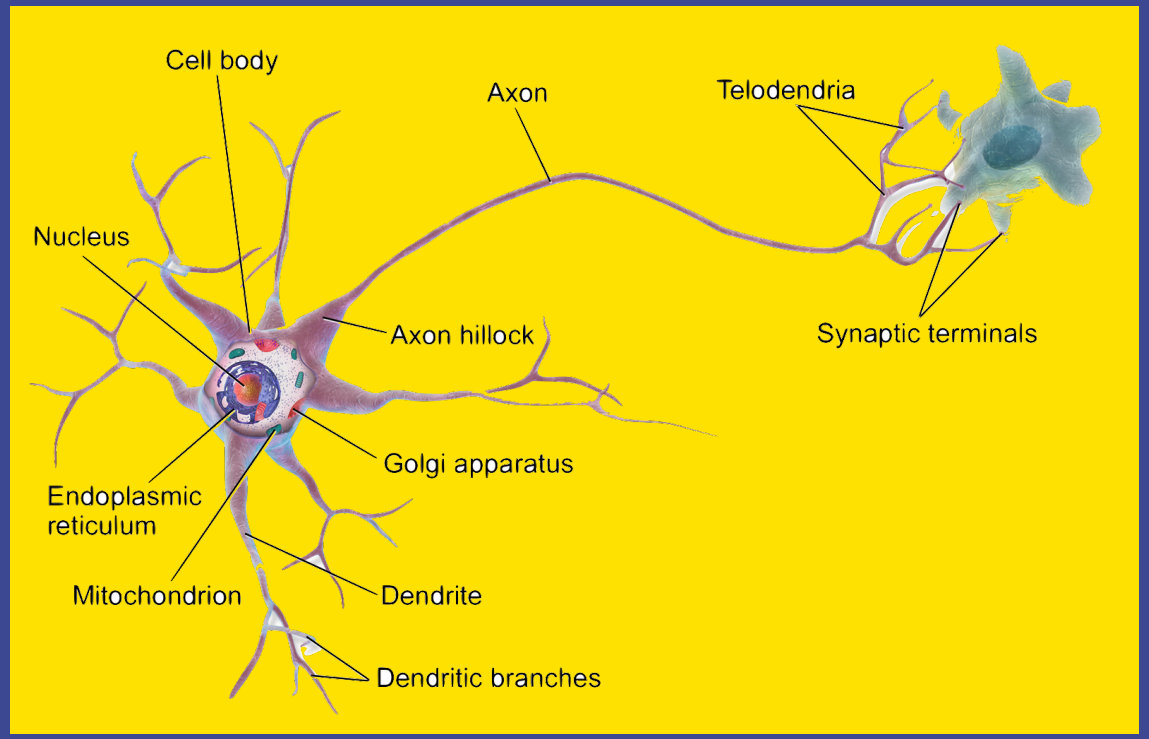
- The naïve Bayes assumption takes all features to be independent given the class label
- We can build classifiers out of a naïve Bayes model using training data
- Smoothing is important in real systems
- Naïve Bayes models tend to perform well when the features are homogeneous

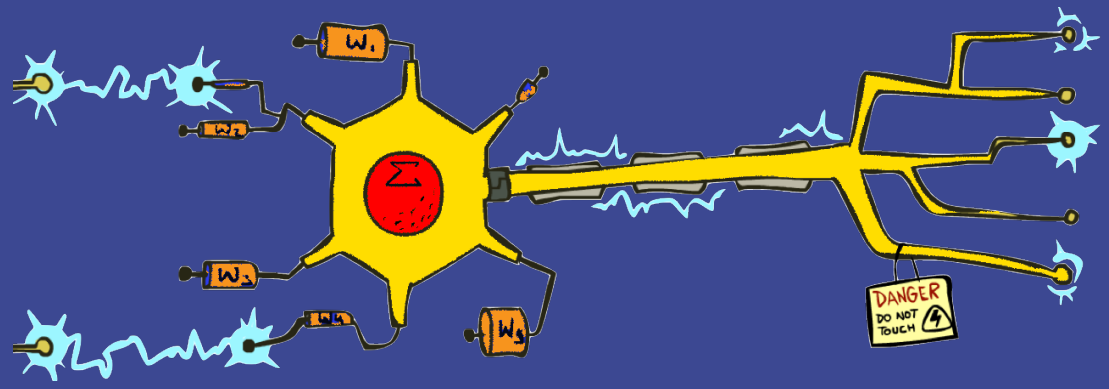
Perceptron



Some (Simplified) Biology

- Very loose inspiration: human neurons

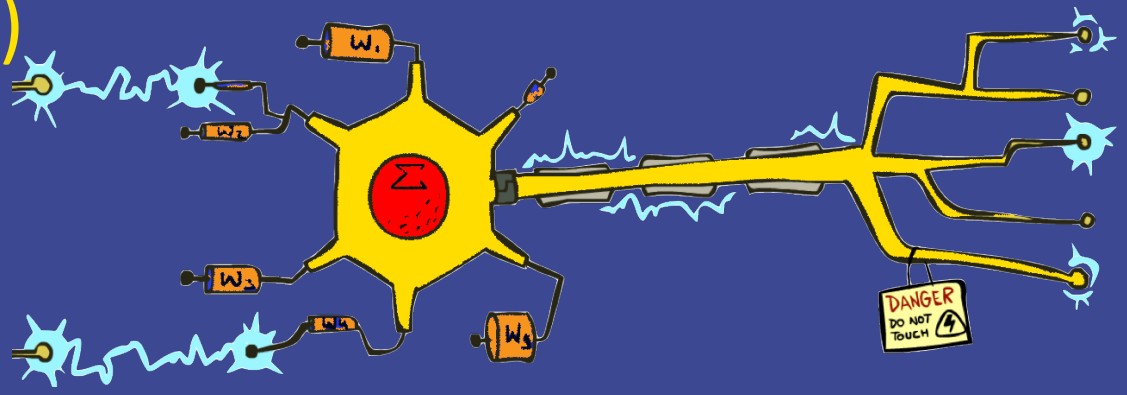




- At birth: 100 billion neurons!

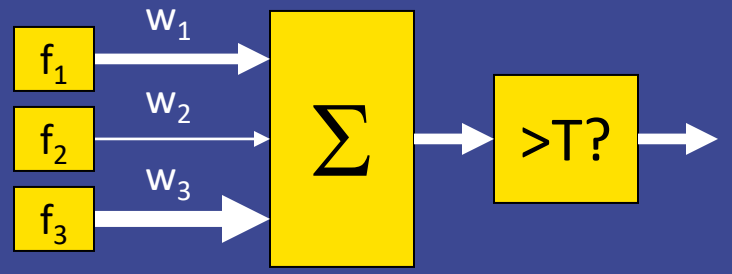
Perceptron: A Linear Classifier

- Inputs are feature values (vectors)
- Each feature has a weight
- Sum is the activation



$$activation_w(x) = \sum_i w_i \cdot fi(x) = w \cdot \mathrm{f(x)}$$

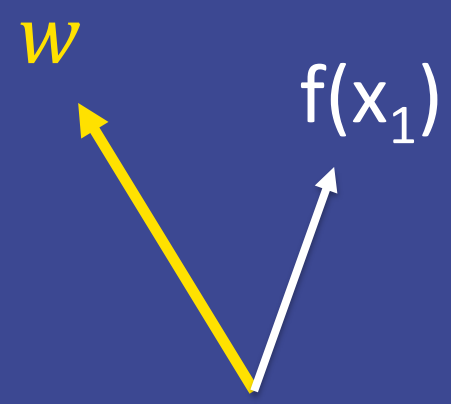
- If the activation is:
  - > threshold T => +1
  - otherwise, no output (0)



Classify

- Binary case: compare features to a weight vector

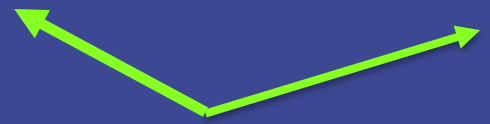
```
# free      : 4
YOUR_NAME   :-1
MISSPELLED  : 1
FROM_FRIEND :-3
```




```
# free      : 2
YOUR_NAME   : 0
MISSPELLED  : 2
FROM_FRIEND : 0
```

For now, let's use 0 as our threshold T

Dot product positive



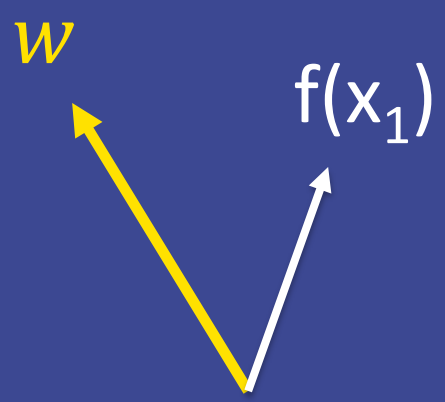
Dot product negative

Dot product 0

Classify

- Binary case: compare features to a weight vector

# free	: 4
YOUR_NAME	:-1
MISSPELLED	: 1
FROM_FRIEND	:-3



# free	: 2
YOUR_NAME	: 0
MISSPELLED	: 2
FROM_FRIEND	: 0

For now, let's use 0 as our threshold T

Dot product: w . $f(x_1)$ = 4 x 2 – 1 x 0 + 1 x 2 - 3 x 0 = 10

Dot product w . f > T => the positive class

Classify

- Binary case: compare features to a weight vector

```
# free       : 4
YOUR_NAME    :-1
MISSPELLED   : 1
FROM_FRIEND  :-3
```

w

Which one is the positive class here?
What class / label does this weight vector support?

A. SPAM
B. HAM

```
: 2
: 0
: 2
D : 0
```

For now, let's use 0 as our threshold T

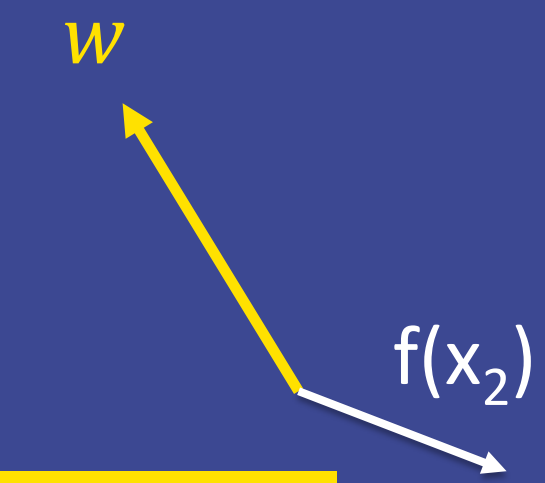
Dot product: w . $f(x_1)$ = 4 x 2 – 1 x 0 + 1 x 2 - 3 x 0 = 10

Dot product w . f > T => the positive class

Classify

- Binary case: compare features to a weight vector

```
# free        : 4
YOUR_NAME     :-1
MISSPELLED    : 1
FROM_FRIEND   :-3
```



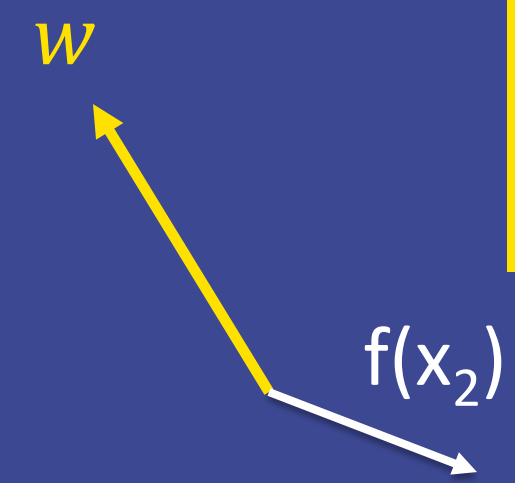

```
# free        : 0
YOUR_NAME     : 1
MISSPELLED    : 1
FROM_FRIEND   : 1
```

iClicker:
What is the dot product: w . f (x_2)?

Classify

- Binary case: compare features to a weight vector

```
# free       : 4
YOUR_NAME    :-1
MISSPELLED   : 1
FROM_FRIEND  :-3
```



What is the predicted label for x_2?

A. SPAM

B. HAM

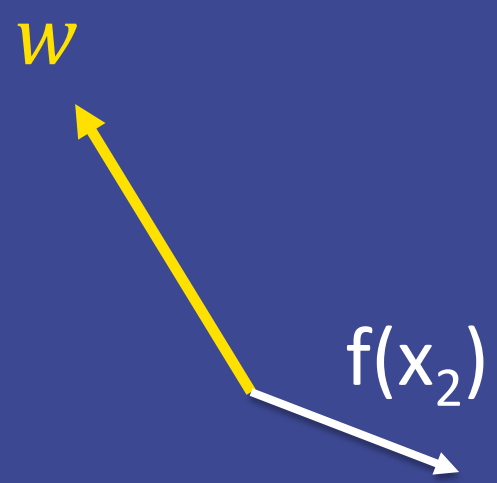
```
YOUR_NAME    : 1
MISSPELLED   : 1
FROM_FRIEND  : 1
```

Dot product: w . $f(x_2)$ = 4 x 0 – 1 x 1 + 1 x 1 - 3 x 1 = -3

Classify

- Binary case: compare features to a weight vector

```
# free       : 4
YOUR_NAME    :-1
MISSPELLED   : 1
FROM_FRIEND  :-3
```




```
# free       : 0
YOUR_NAME    : 1
MISSPELLED   : 1
FROM_FRIEND  : 1
```

Dot product: $w \cdot f(x_2) = 4 \times 0 - 1 \times 1 + 1 \times 1 - 3 \times 1 = -3$

Dot product w . f < T means the negative/other class (HAM)

Threshold vs Bias

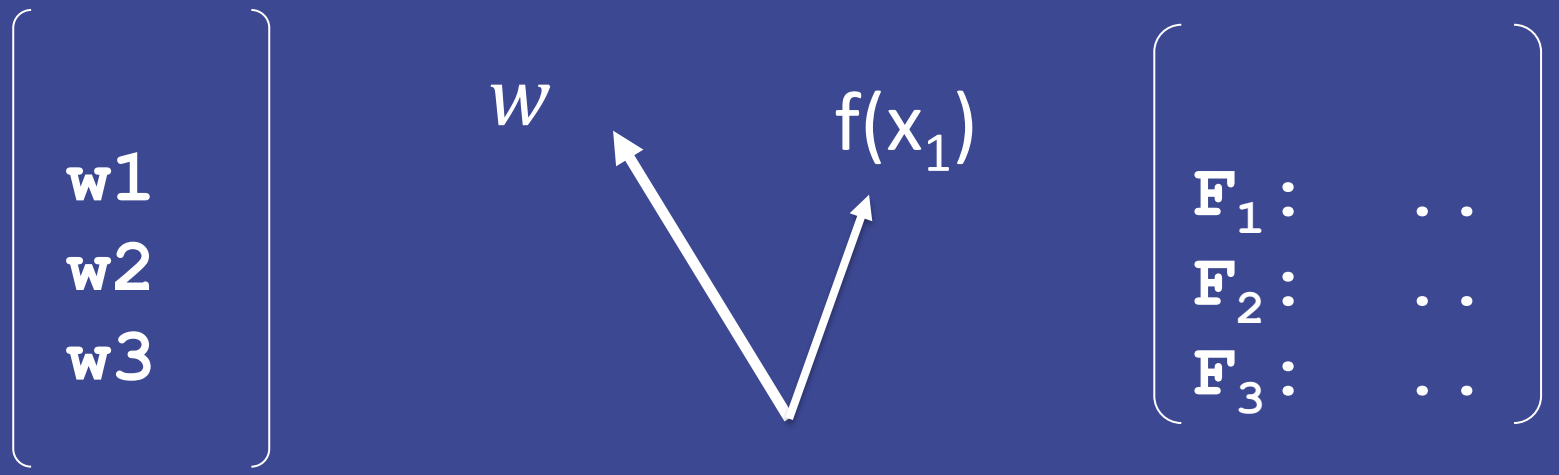
if w . f(x) > T: => +1 else: 0

if w . f(x) - T > 0: => +1 else: 0

if w . f(x) - (- BIAS) > 0: => +1 else: 0

if w . f(x) + BIAS > 0: => +1 else: 0

BIAS = -T



Threshold vs Bias

if w . f(x) > T: => +1 else: 0

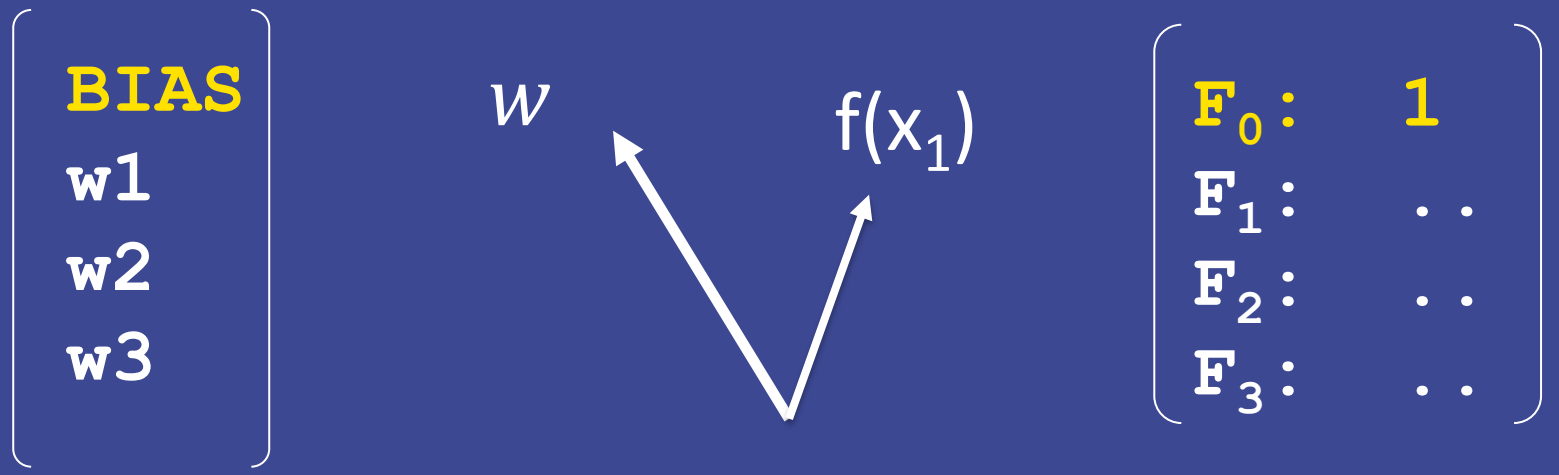
if w . f(x) - T > 0: => +1 else: 0

BIAS = -T

if w . f(x) - (- BIAS) > 0: => +1 else: 0

if w . f(x) + BIAS > 0: => +1 else: 0

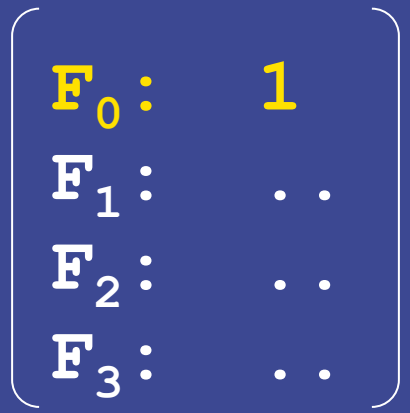
The bias does not depend on any input value.



Threshold vs Bias

if w' . f'(x) > 0: => +1 else: 0

w'

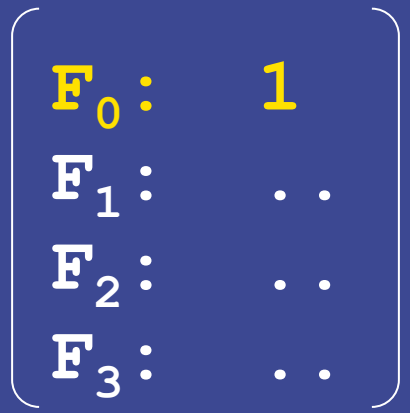


f'(x_1)

Threshold vs Bias

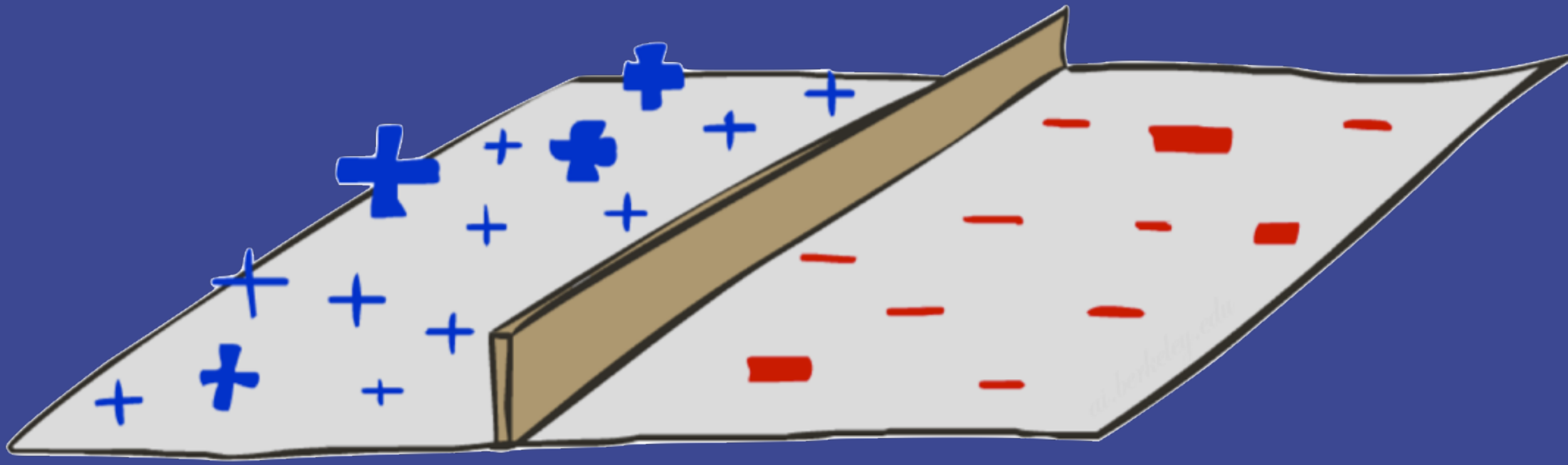
if w . f(x) > 0: => +1 else: 0

$w(x_1)$



$f(x_1)$

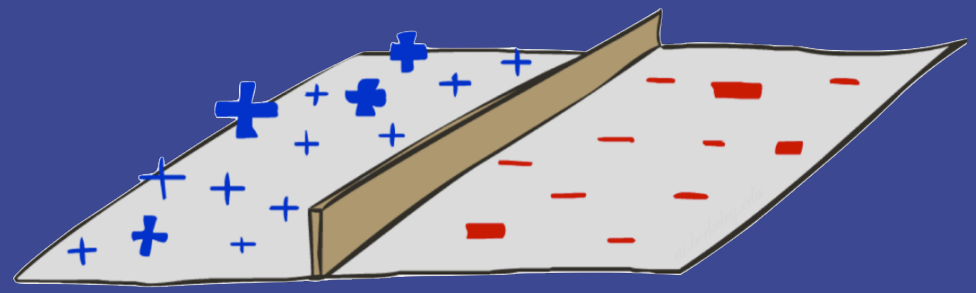
Decision Rules

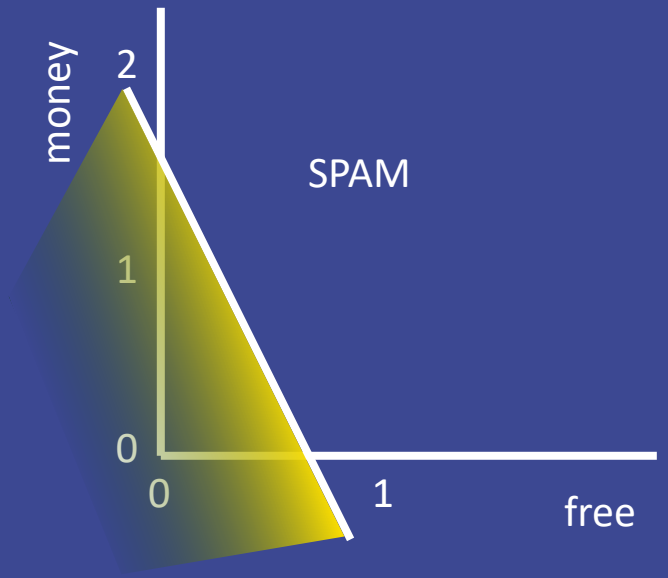


Binary Decision Rule

In the space of feature vectors

- Examples are points
- A weight vector corresponds to a decision boundary that is a hyperplane (a line in 2D, a plane in 3D)
- One side corresponds to f.w > 0 => SPAM
- Other corresponds to f.w <= 0 => HAM





$$\begin{pmatrix} \texttt{BIAS} & : & -3 \\ \texttt{free} & : & 4 \\ \texttt{money} & : & 2 \end{pmatrix}$$

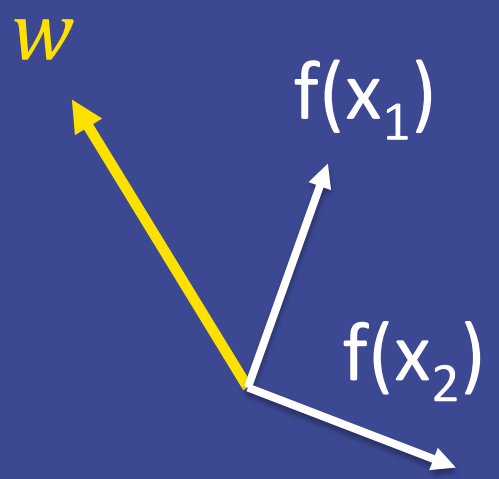
HAM

f . w = 0

Classification?

- classification: predict the label based on the features and the weight vector

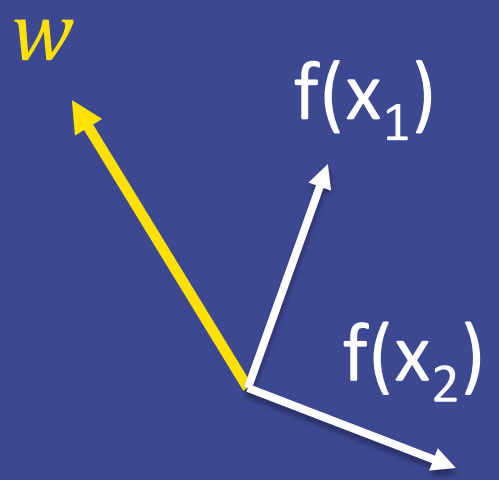
```
BIAS
w1
w2
w3
w4
```



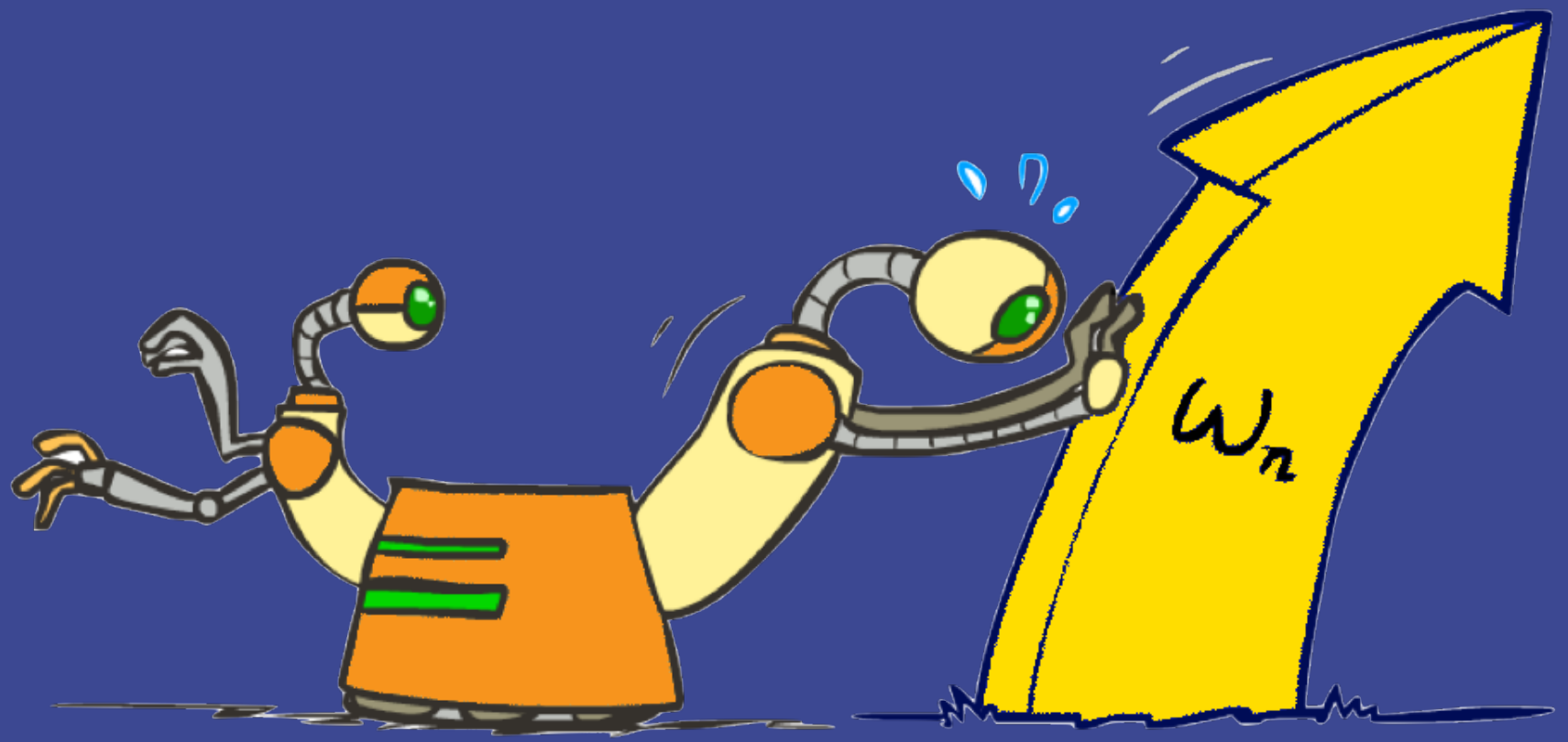
Learning?

- Learning: figure out the weight vector from labeled examples

```
BIAS
w1
w2
w3
w4
```

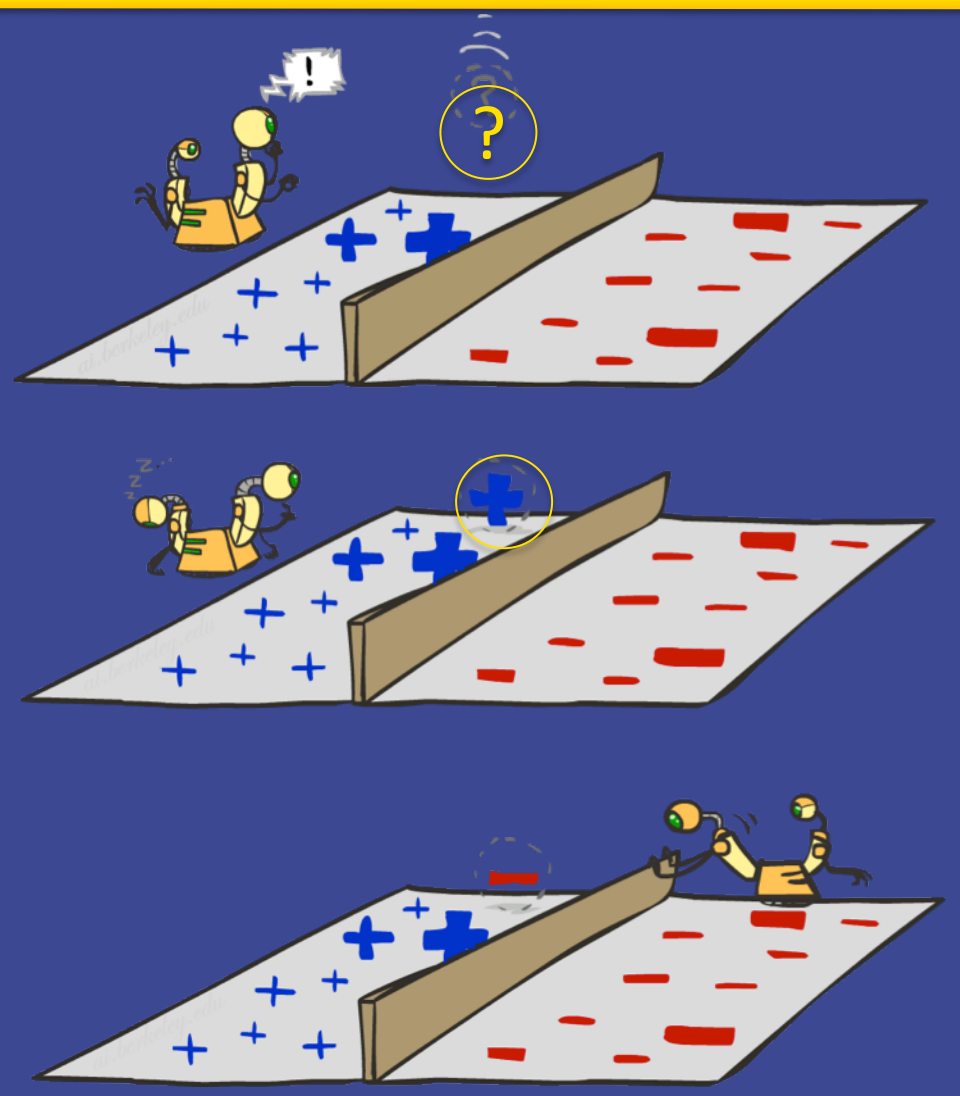


Weight Updates



Learning: Binary Perceptron

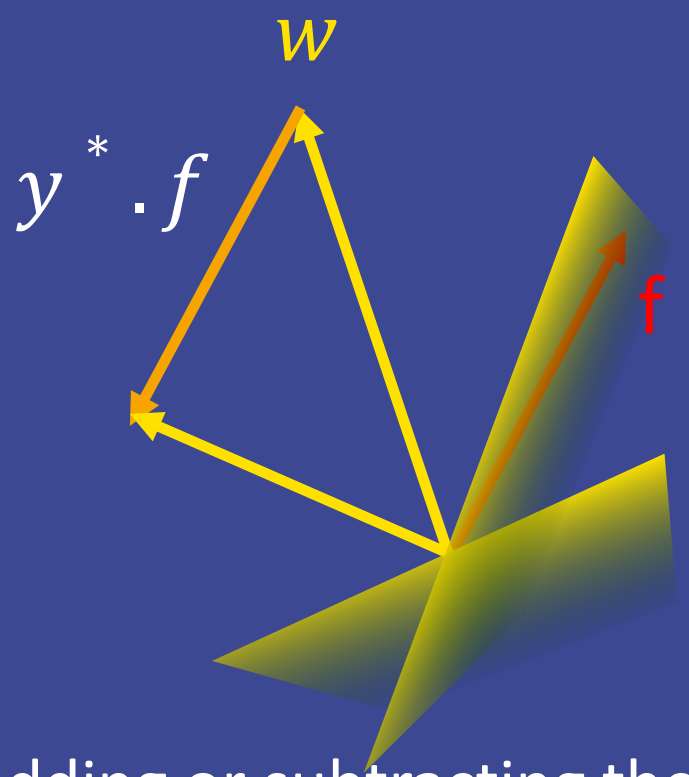
- Start with arbitrary weights
- For each training instance:
  - Classify with current weights
  - If correct, no change!
  - If wrong: adjust the weight vector
- Repeat until no more significant changes (iterations)



Learning: Binary Perceptron

- Start with weights = 0
- For each training instance:
  - Classify with current weights

$$y = \begin{cases} \text{Positive class if } w.f(x) > 0 \\ \text{Other class if } w.f(x) <= 0 \end{cases}$$



  - y*: training label
  - If correct (i.e., y=y*), no change!
  - If wrong: adjust the weight vector by adding or subtracting the feature vector: $w = w \pm f$ (subtract if y* is the negative class)
- Repeat until no more significant changes (iterations)

Reminders

- Homework 6 due at 5PM
- QoW on Tuesday